UNIVERSIDAD MAYOR, REAL Y PONTIFICIA DE SAN FRANCISCO XAVIER DE CHUQUISACA

FACULTAD DE CIENCIAS Y TECNOLOGIA

CARRERA DE CIENCIAS DE LA COMPUTACIÓN



Practica Microservicios

Universitario: Pedro Jhoel Antonio Magne Ordoñez

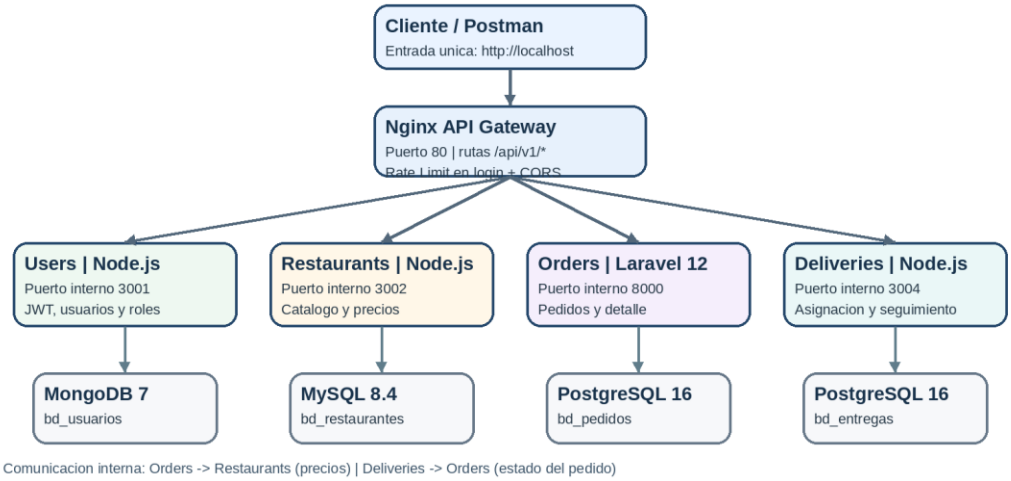
Materia: COM350

Grupo: N°1

SUCRE – BOLIVIA

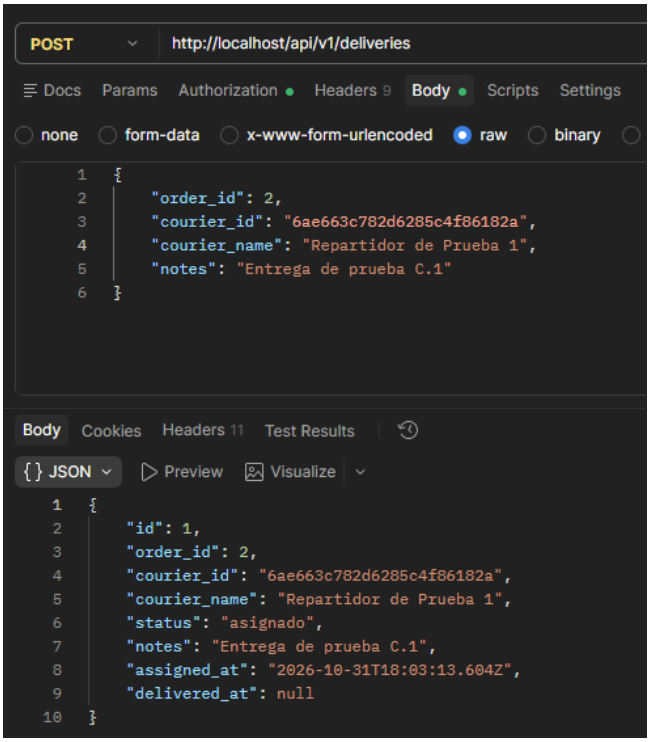
2026

1. **Arquitectura desplegada**

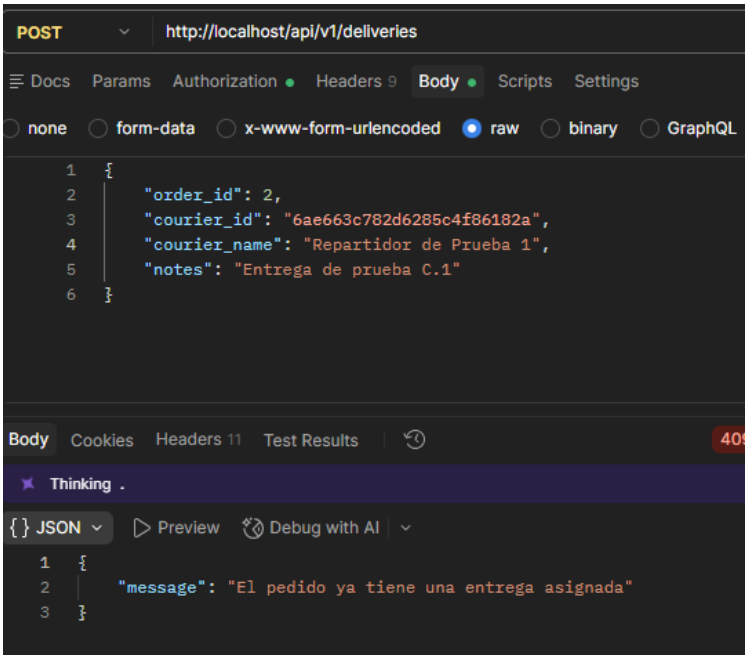


2. **Casos de prueba**

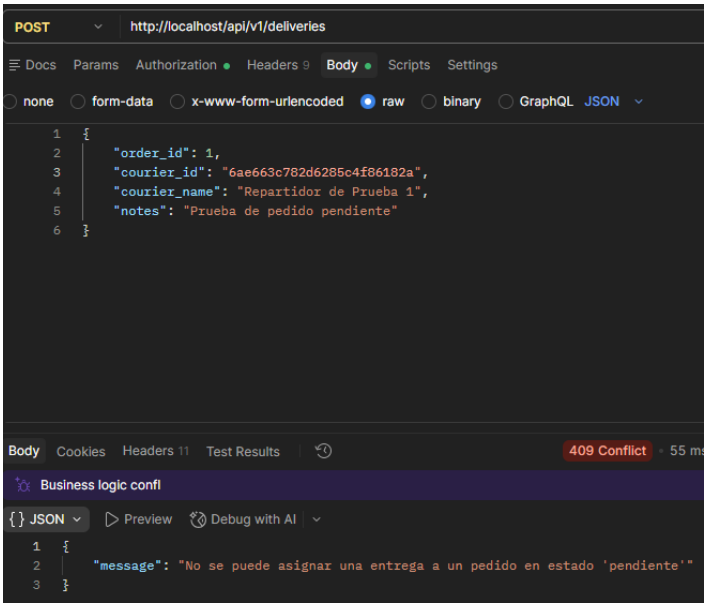
Admin asigna el pedido 1 (en_preparacion) al repartidor



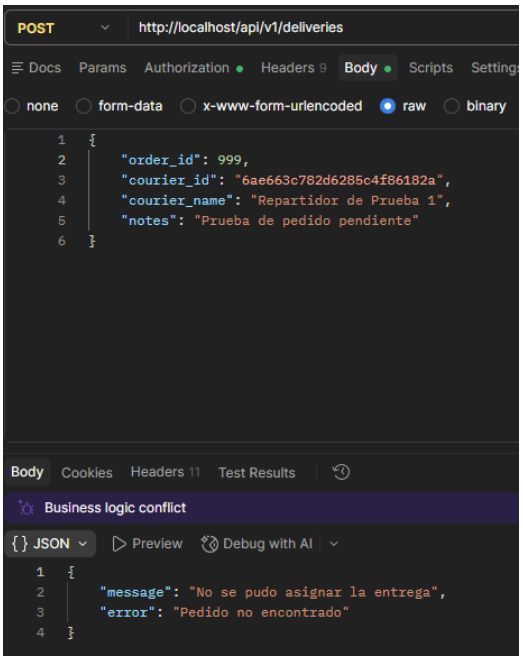
Admin asigna de nuevo el pedido 1



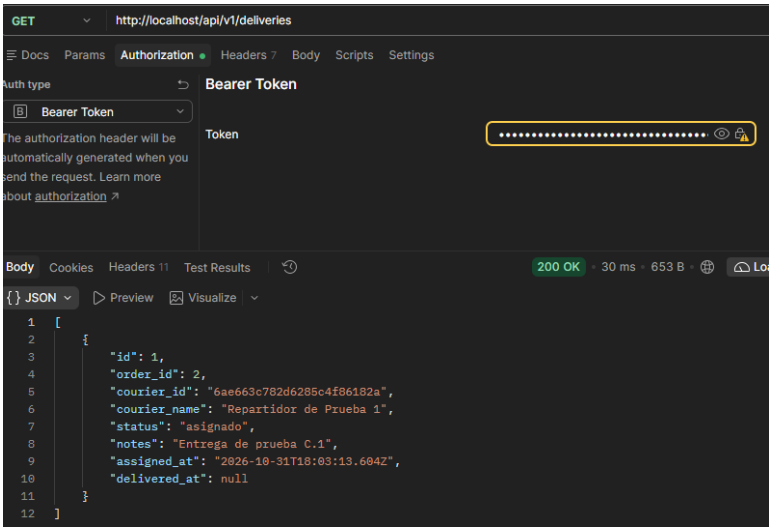
Admin asigna un pedido pendiente



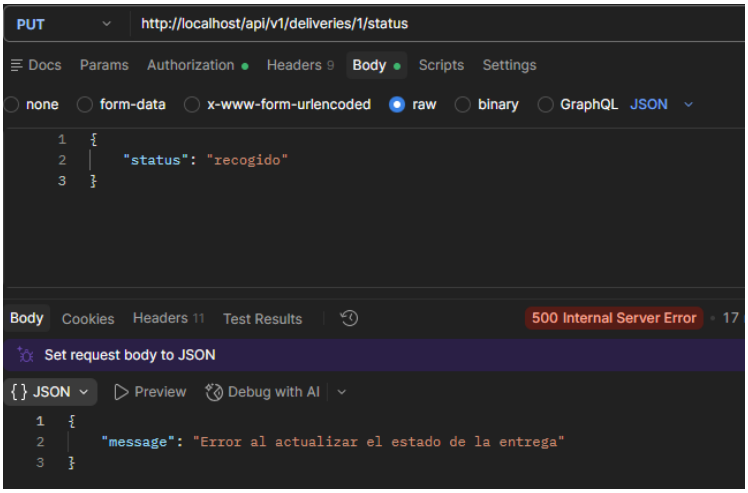
Admin asigna el pedido 999



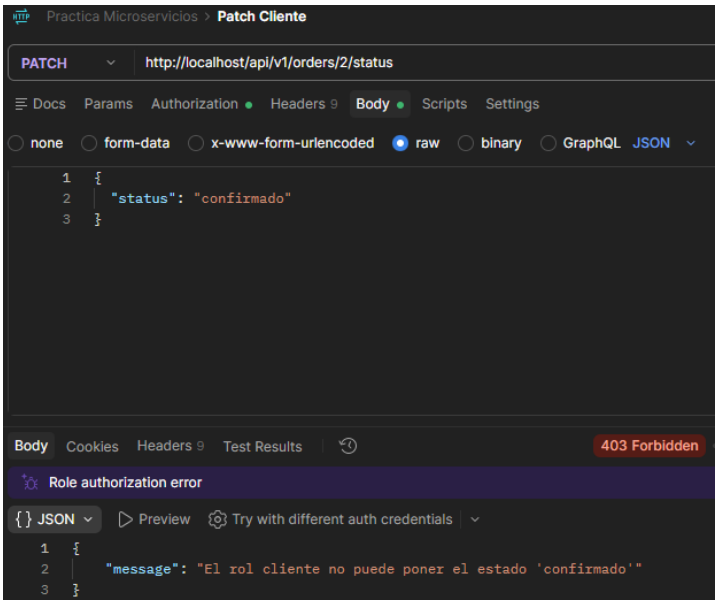
Repartidor lista GET /deliveries



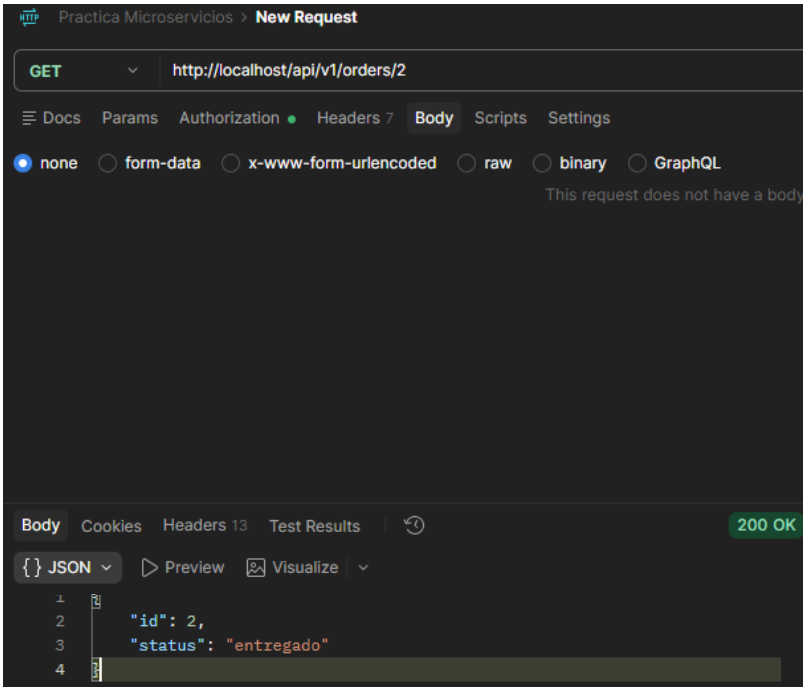
Otro repartidor cambia el estado de esa Entrega



Repartidor intenta volver a recogido



GET /api/v1/orders/1 con el cliente



3. Observaciones de la práctica

Observación 1: ¿Por qué los puertos 3001, 3002, 8000 y 3004 siguen publicados en el host si el cliente solo debería usar el 80? ¿Qué pasaría si se eliminan las líneas ports de los microservicios?

Los puertos están publicados para permitir acceder directamente a los microservicios desde el host, facilitando las pruebas y la depuración. Sin embargo, en la arquitectura final el cliente debería comunicarse únicamente mediante Nginx, que centraliza el acceso y el enrutamiento.

Si eliminamos los ports de los microservicios, estos seguirían comunicándose entre sí mediante la red interna de Docker, pero ya no serían accesibles directamente desde el host. El cliente seguiría accediendo al sistema por el puerto 80 de Nginx.

Observación 2: Los tres scripts solo corren cuando el volumen está vacío. Si modifica un script después del primer up, ¿qué comando debe ejecutar para que se aplique y qué se pierde?

Se debe ejecutar:

- docker compose down -v
- docker compose up -d --build

El parámetro -v elimina los volúmenes persistentes, haciendo que las bases de datos se inicialicen nuevamente y ejecuten sus scripts. Como consecuencia, se pierden los datos almacenados en MongoDB, MySQL y PostgreSQL, incluyendo usuarios registrados, restaurantes, pedidos y entregas.

Observación 3: En User.js el toJSON elimina el campo password. ¿Por qué es preferible hacerlo en el modelo y no en cada ruta?

Porque permite centralizar la protección de los datos sensibles. Al eliminar password desde el modelo, cualquier ruta que devuelva un usuario mediante JSON evitará exponer su contraseña, sin necesidad de repetir esa lógica en cada controlador. Esto reduce errores y mejora la seguridad del sistema.

Observación 4 user_**id y restaurant_id no son claves foráneas y además se copia user_name y restaurant_name. Explique la decisión en términos del patrón base de datos por servicio y qué problema de consistencia introduce.**

Esto se hace porque cada microservicio es propietario de su propia base de datos y no se deben establecer dependencias mediante claves foráneas entre servicios diferentes. Los identificadores permiten relacionar lógicamente los datos, mientras que los nombres copiados permiten consultar el pedido sin depender constantemente de los otros microservicios.

El problema de consistencia es que, si posteriormente cambia el nombre de un usuario o restaurante, la información almacenada en el pedido podría quedar desactualizada. También se pierde la integridad referencial que ofrecería una clave foránea.

Observación 5: Si el servicio restaurants está caído, ¿qué código HTTP devuelve POST /api/v1/orders y por qué? ¿Cómo lo mejoraría?

Devuelve HTTP 500 Internal Server Error, porque Laravel intenta comunicarse con restaurants mediante una petición HTTP y, si el servicio no responde, se genera una excepción de conexión que no está controlada en el controlador. La petición no llega a completar la creación del pedido.

Se podría mejorar capturando esa excepción y devolviendo HTTP 503 Service Unavailable o 502 Bad Gateway. También se podrían implementar reintentos para fallos temporales, un circuit breaker para evitar llamadas repetidas a un servicio caído y una caché de precios, teniendo cuidado de no utilizar precios desactualizados para confirmar pedidos.

4. **C.4**

Pregunta 1: Un plato cambia de precio después de que el pedido fue creado. ¿Qué muestra GET /orders/:id? Justifique por qué se guardó unit_price en order_items.

El endpoint muestra el precio que tenía el plato al momento de crear el pedido, no el precio actualizado del restaurante. Esto sucede porque unit_price se guarda directamente en order_items.

Se realiza así para conservar el historial de la compra y evitar que los cambios posteriores en el catálogo modifiquen el total de pedidos anteriores. De esta manera, el importe acordado con el cliente permanece constante.

Pregunta 2: ¿Qué ocurre si deliveries actualiza su tabla pero la llamada PATCH a orders falla porque el servicio está caído? Proponga una solución basada en Saga u Outbox con un broker de mensajes.

Se produce una inconsistencia entre los servicios, la entrega puede aparecer como entregada en la base de datos de Deliveries, mientras que el pedido mantiene su estado anterior en Orders. En nuestra implementación, Deliveries responde con HTTP 502 cuando falla la sincronización.

Una solución sería utilizar el patrón Transactional Outbox con un broker de mensajes como RabbitMQ. Cuando se actualiza la entrega, se registra también un evento en una tabla outbox, dentro de la misma transacción local. Después, un proceso publica ese evento en el broker y Orders lo consume para actualizar el pedido. Si Orders está caído, el evento puede reintentarse posteriormente hasta completar la sincronización.

Pregunta 3: Compare validar el JWT en cada microservicio con validarlo una sola vez en Nginx.

Validar el JWT en cada microservicio permite que cada servicio compruebe por sí mismo la autenticidad y vigencia del token, además de aplicar sus propias reglas de autorización. Esto mejora la independencia y evita depender exclusivamente del Gateway. Sin embargo, requiere repetir la lógica de validación y mantener el secreto compartido correctamente configurado.

Validarlo únicamente en Nginx centraliza la autenticación, reduce la duplicación de código y facilita la aplicación de políticas comunes. Sin embargo, los servicios internos podrían quedar desprotegidos si alguien consigue acceder directamente a ellos o si se configura incorrectamente la confianza en los encabezados enviados por el Gateway.

Por eso, validar en cada microservicio ofrece una protección adicional, aunque requiere más configuración.

Pregunta 4: Dos servicios comparten el contenedor PostgreSQL con bases distintas. ¿Se cumple el patrón base de datos por servicio? ¿Qué se gana y qué se pierde frente a dos contenedores separados?

Sí, se cumple el patrón porque cada microservicio posee una base de datos independiente y administra exclusivamente sus propios datos, aunque ambas bases estén en el mismo servidor PostgreSQL.

La ventaja es que se reducen los recursos utilizados y la complejidad de administrar contenedores separados. La desventaja es que ambos servicios comparten los recursos del servidor, su disponibilidad y sus tareas de mantenimiento. Si PostgreSQL falla, los dos microservicios pueden verse afectados.

Pregunta 5: Mida con docker stats la memoria de los cuatro microservicios. ¿Cuál consume más y por qué? ¿Cómo influye la elección de tecnología en el costo de operar microservicios políglotas?

Estos son los resultados de docker stats:

Microservicio	Memoria utilizada	Memoria %
Users (Node + MongoDB)	60.96 MiB	0.62 %
Orders (Laravel + PostgreSQL)	59.23 MiB	0.61 %
Restaurants (Node + MySQL)	46.91 MiB	0.48 %
Deliveries (Node + PostgreSQL)	44.23 MiB	0.45 %

El microservicio que presenta el mayor consumo de memoria es Users, con 60.96 MiB (0.62 %), seguido de Orders con 59.23 MiB (0.61 %). En cambio, Deliveries presenta el menor consumo, con 44.23 MiB (0.45 %).

Estas diferencias pueden deberse a las tecnologías y dependencias utilizadas por cada servicio. Por ejemplo, Users utiliza Node.js con Mongoose para comunicarse con MongoDB, mientras que Orders utiliza Laravel y PHP. Sin embargo, la medición por sí sola no permite determinar exactamente qué componente provoca el mayor consumo.

La arquitectura de microservicios políglotas permite utilizar distintas tecnologías según las necesidades de cada servicio, pero también implica mayores costos de mantenimiento, configuración, monitoreo y recursos. Por eso, es importante evaluar el consumo de cada tecnología antes de desplegar el sistema.

Una aclaración para el informe: estos valores corresponden a una medición puntual con --no-stream, no a una prueba de carga. Los consumos podrían variar al recibir más solicitudes.